INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN NORMANDIE

Département ITI 3

Rapport du PI Robot

Projet Intégratif Smart Robot

Étudiants

GRONDIN	Yann	yann.grondin@insa-rouen.fr
KAMARA	Mohamed	mohamed.kamara@insa-rouen.fr
FAKHOURY	Zouhair	zouhair.fakhoury@insa-rouen.fr
SAVARY	Théo	theo.savary@insa-rouen.fr
SAAD DJABALLAH	Seif-Din	seif-din.saad_djaballah@insa-rouen.fr

Projet GitLab

21 octobre 2026

Table des matières

1 Introduction **2**

2 Partie électronique **3**
2.1 Choix de câblage et décisions techniques 3
2.2 Bilan de puissance et autonomie 5
2.3 Description des composants 5
2.4 Organigramme de la démarche 5
2.5 Difficultés et améliorations 7

3 Partie informatique **8**
3.1 Analyse 8
3.2 Conception Préliminaire 8
3.3 Conception détaillée et algorithmes 10
3.4 Programme principal du robot 12
3.5 Programme de gestion des intersections 13
3.6 Programme de suivi de ligne 13
3.7 Implémentation et tests 13

4 Conclusion et matrice de contribution **16**
4.1 Résumé des contributions 16

5 Annexe du rapport **17**

1 Introduction

Le projet d'intégration PI Smart Robot consiste à concevoir un robot mobile capable de parcourir le plus vite possible une ville modélisée par une grille de cases chacune reliée par des lignes noires. Le robot est positionné au départ sur une pastille rouge avec une orientation connue (Nord, Est, Sud ou Ouest). Il doit visiter toutes les pastilles vertes en émettant un "bip" à chaque point de passage, puis revenir à son point de départ. La description de la ville est fournie dans un fichier texte (largeur l, case et orientation initiales, liste des passages entre cases, liste des cases à visiter, terminée par un point). La commande du robot se fait à l'aide d'un flux d'ordres élémentaires `AV` (avancer jusqu'au prochain croisement), `TG` (tourner à gauche) et `TD` (tourner à droite), et s'achève par un point.

Le jour de la démonstration, un script bash lance deux programmes communicants : le premier lit le fichier de description et produit sur sa sortie standard la séquence d'ordres `AV`, `TG`, `TD`, `...` Le second lit cette séquence, interprète les ordres et pilote le robot. Le robot est comparé aux autres équipes grâce au temps d'exécution total.

Toutes les équipes disposent des mêmes composants, cependant le choix revient aux équipes quant à leur utilisation.

Tout d'abord, nous présenterons la partie électronique, elle détaillera la conception et la réalisation du robot. Puis, nous verrons la partie informatique qui poursuivra sur la mise en oeuvre d'un cycle en V ainsi que les algorithmes non triviaux utilisés. Enfin, nous conclurons par une synthèse des contributions individuelles, des perspectives d'amélioration et les difficultés rencontrées.

2 Partie électronique

Dans cette première partie, nous allons décrire les différents choix réalisés lors de la conception du robot. Puis, nous ferons une analyse de la puissance et de l'autonomie de celui-ci. Ensuite, nous détaillerons rapidement les composants utilisés avant de finir sur l'organigramme de la démarche.

2.1 Choix de câblage et décisions techniques

2.1.1 Alimentation et sécurité des moteurs

Pour débuter le projet, la priorité était d'assurer le contrôle des moteurs tout en respectant la consigne de sécurité : pouvoir arrêter le robot via un interrupteur physique.

Le choix de câblage a consisté à relier le bloc d'alimentation (piles 9V) à l'interrupteur, ce dernier étant ensuite relié au driver moteur (pont en H). De cette manière, si l'interrupteur est en position ouverte, le circuit est physiquement coupé : le driver n'est plus alimenté et les moteurs s'arrêtent instantanément, garantissant un arrêt d'urgence matériel.

2.1.2 Stratégie de suivi de ligne

L'objectif principal était de permettre au robot de suivre une ligne au sol. Initialement, nous avions envisagé de placer deux capteurs de part et d'autre de la ligne. Finalement, nous avons opté pour un placement de deux capteurs directement **sur** la ligne, selon la logique algorithmique suivante :

Si l'un des capteurs détecte une sortie de ligne (changement de réflectance), il faut compenser en activant le moteur opposé pour redresser la trajectoire.

Exemple : Soit Cap_G et Cap_D les capteurs gauche et droit, et M_G, M_D les moteurs respectifs :

- Si Cap_G est activé $\Rightarrow$ le robot s'éloigne de la ligne vers la gauche.
- Action corrective : $M_G = 1$ et $M_D = 0$ pour pivoter vers la droite.

2.1.3 Analyse du comportement dynamique

Nous avons observé que cette action de redressement, selon le rapport de cycle (PWM) appliqué aux moteurs, peut avoir un impact néfaste. Un redressement trop brusque entraîne une déviation du robot dans la direction opposée (dépassement de consigne). Le robot est alors contraint de se redresser à nouveau, créant un phénomène d'oscillation permanent, ou "zig-zag", même sur une ligne droite.

2.1.4 Alimentation des capteurs et intersections

Les capteurs de détection de ligne sont alimentés en **3,3 V**, conformément à leur fiche technique. Cette tension est prélevée directement sur les broches d'alimentation du Raspberry Pi 4B (qui dispose de deux pins 3,3 V et deux pins 5 V).

2.1.5 Protocoles de communication

— **I2C :** L'écran LCD et le capteur RGB (TCS3472) utilisent ce protocole via les broches **SDA** (Data) et **SCL** (Clock), communiquant sur les fronts montants de l'horloge.

— **Buzzer :** Le buzzer est relié directement à une broche GPIO du Raspberry Pi, émettant un signal sonore dès qu'une condition spécifique (détection d'une pastille) est remplie.

2.1.6 Communication et détection d'intersections

— **Intersections :** Deux capteurs latéraux supplémentaires ont été ajoutés pour détecter les lignes perpendiculaires et d'envoyer le signal au Raspberry Pi pour déclencher une séquence de rotation.

— **Protocoles :** L'écran LCD et le capteur RGB partagent le bus **I2C** (SDA/SCL). Le buzzer est piloté par un signal numérique direct (GPIO).

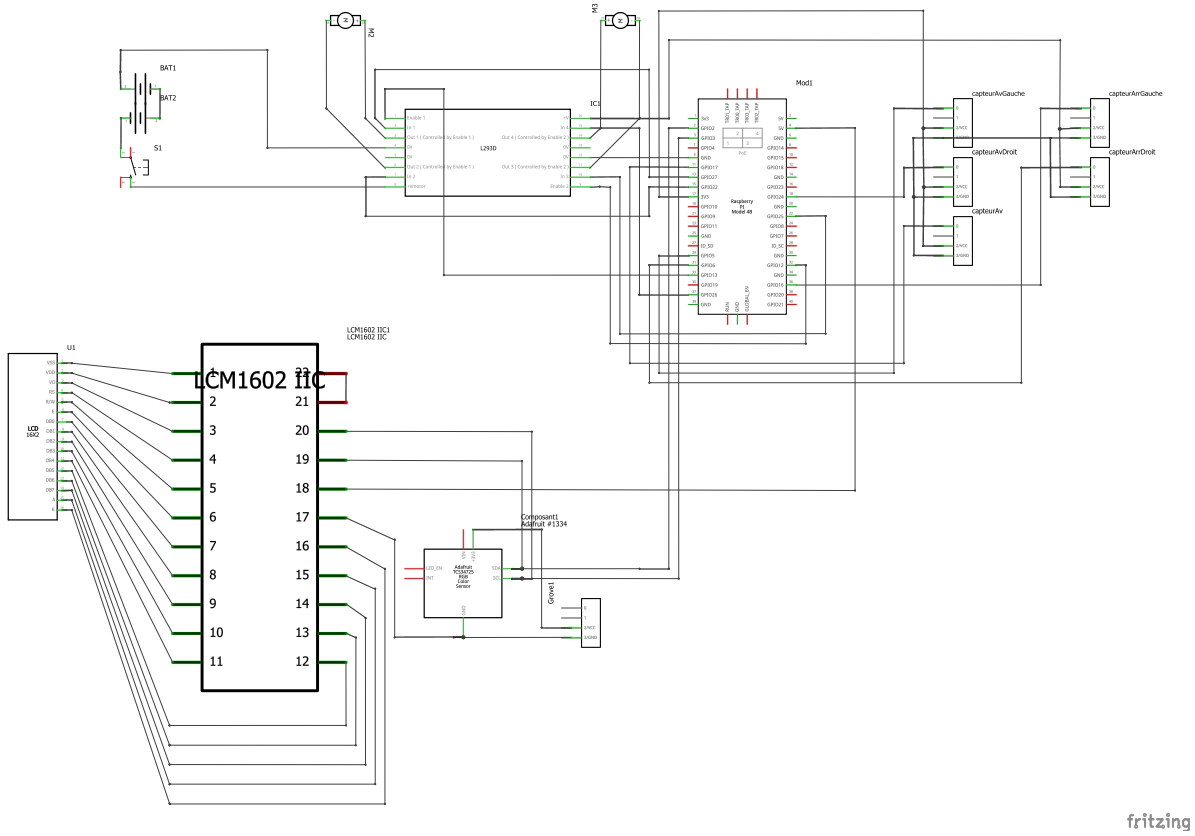


FIGURE 1 – Schéma électrique du robot

Composants	Calcul Puissance Absorbée (P_a)	Bilan Total
Raspberry PI 4 B	$5\ \text{V} \times 1\ \text{A}$	5 W
TCRT5000 (x5)	$5 \times (3,3\ \text{V} \times 60\ \text{mA})$	0,99 W
Moteurs DC (x2)	$2 \times (1\ \text{A} \times 4,5\ \text{V})$	9 W
TCS3472	$3,6\ \text{V} \times 235\ \mu\text{A}$	0,00085 W
LCD I2C 16x2	$(5\ \text{V} \times 35\ \text{mA}) + (5\ \text{V} \times 25\ \text{mA})$	0,3 W
Buzzer Grove	$3,3\ \text{V} \times 20\ \text{mA}$	0,066 W
TOTAL	**Somme des consommations**	**15,36 W**

TABLE 1 – Bilan de puissance corrigé du robot

2.2 Bilan de puissance et autonomie

2.2.1 Tableau récapitulatif de la consommation

2.2.2 Analyse de l'autonomie

Le système utilise deux sources d'énergie distinctes : On alimente notre Raspberry Pi 4 B avec une batterie portative délivrant 10000 mAh, et on alimente le pont en H grâce à deux piles (env.) 8,5-9 V. Pour déterminer l'autonomie de notre robot, on suppose les piles et la batterie chargées. De plus, nous prenons en compte le fait que la batterie portative n'a pas un rendement parfait donc elle délivrera pour l'étude 9000 mAh.

1. **Batterie Portative :** Capacité utile : $10000\ \text{mAh} \times 0,9 = 9000\ \text{mAh}$. Courant consommé (estimé) : 1140 mA.
$$Autonomie_{Elec} = \frac{9000}{1140} \approx 7,89\ \text{h}$$
2. **Bloc Piles (Moteurs) :** Capacité estimée : 6800 mAh. Courant : 2000 mA.
$$Autonomie_{Moteurs} = \frac{6800}{2000} = 3,4\ \text{h}$$

2.3 Description des composants

— **Raspberry Pi 4 B (x1)** : Micro-ordinateur central faisant office de cerveau pour la gestion des composants et du traitement logique.

— **Capteurs infrarouges TCRT5000 (x5)** : Capteurs photosensibles mesurant la réflectance de la surface pour la détection de ligne.

— **Moteurs DC (x2)** : Moteurs à courant continu (type RM2 avec pignon) pour la propulsion du châssis.

— **Capteur RGB TCS3472 (x1)** : Capteur de couleur numérique identifiant la teinte via les composantes rouge, vert et bleu (protocole I2C).

— **LCD I2C 16x2 + Module Backpack** : Interface d'affichage pour la visualisation des données en temps réel.

— **Buzzer Grove (x1)** : Avertisseur sonore piézoélectrique pour les signaux d'état ou les alertes.

2.4 Organigramme de la démarche

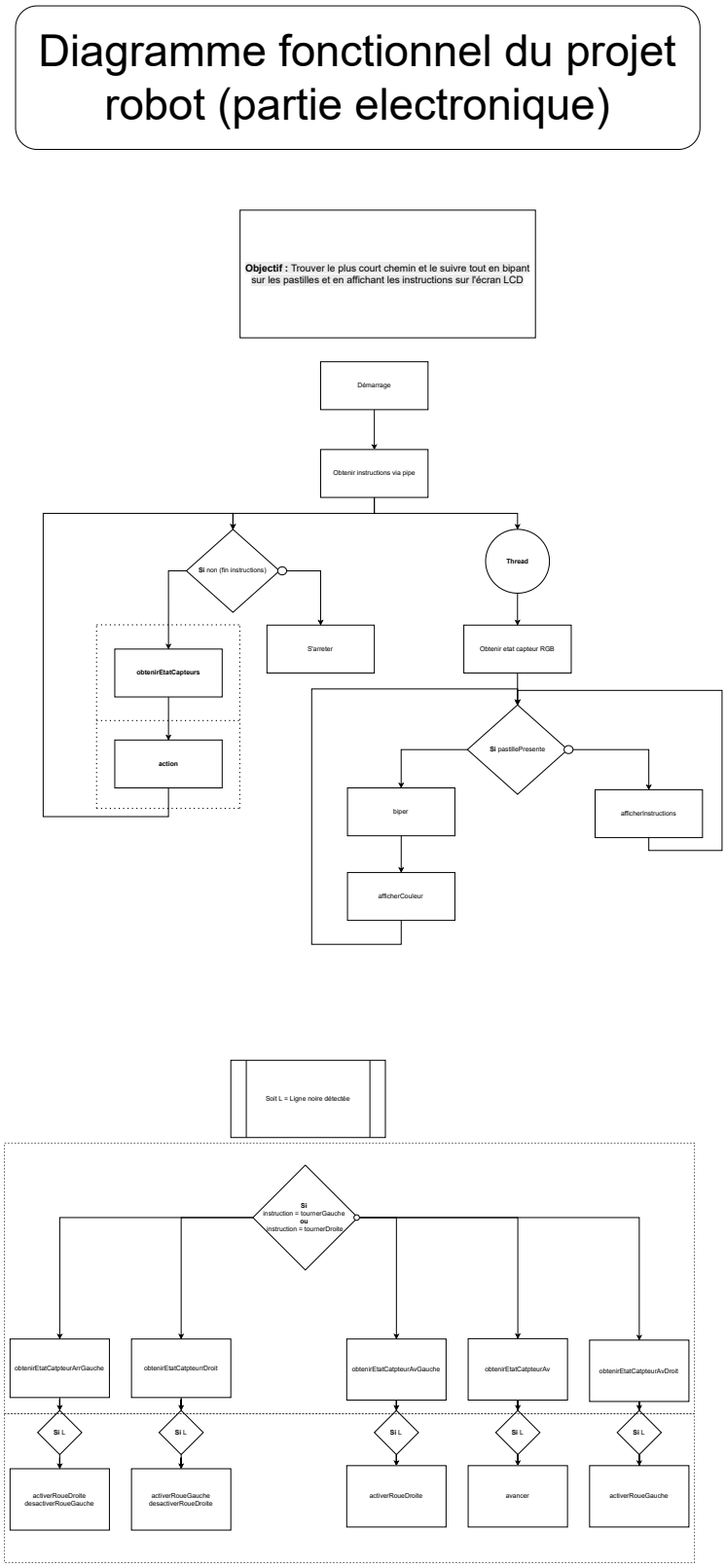


FIGURE 2 – Schéma fonctionnel répondant au cahier des charges.

2.5 Difficultés et améliorations

— Tout d'abord, le couple des moteurs utilisés était différent, ce qui entraînait des écarts de vitesse lors des déplacements linéaires. Ce problème peut être réglé en ajustant les valeurs données aux pins Enable PWM du contrôleur des moteurs, mais le résultat obtenu n'est pas parfait. Cela influence donc la précision globale du robot. L'ajout d'un coefficient n'est qu'une solution temporaire et n'a pas pas été conservé dans la version finale.

— Le robot présentait un comportement irrégulier. Parfois, les corrections de trajectoire ou la détection d'intersection étaient retardées par rapport à l'état réel, visible, des capteurs. La source de ce problème est difficile à identifier, car elle peut provenir soit de la partie électrique (branchement, calibrage, saturation des GPIO), soit de la partie informatique (temps de traitement trop long, saturation du CPU et/ou latences du thread). Ce délai a nécessité des ajustements (comme le fait de tourner dans la direction opposée lorsqu'une intersection est détectée).

— Nous avons pu observer que le robot détectait parfois des intersections en permanence, même lorsqu'il n'y avait pas de croisements. Cette détection erratique pourrait être due à une surcharge du Raspberry, causée par le thread de lecture du capteur RGB ou encore par une mauvaise calibration des capteurs. Ces raisons pourraient également expliquer le décalage entre la lecture des capteurs et les actions du robot, d'où son comportement imprévisible.

2.5.1 Améliorations

— **Gestion logicielle :**
  — Optimiser la détection des intersections et instaurer un protocole de calibration et de placement des capteurs rigoureux et strict.
  — Implémenter des interruptions matérielles pour traiter les signaux des capteurs sans polling.
  — Utiliser un régulateur **PID** pour supprimer l'effet de zig-zag.
  — Introduire une priorisation des processus entre capteur RGB et gestion du robot, ou instaurer une communication entre les deux, par exemple en effectuant la détection du capteur RGB uniquement lorsque le robot est en mode suivi de ligne.
  — Optimiser le thread du capteur RGB afin d'en réduire l'impact sur les ressources du Raspberry et ainsi éviter sa surcharge.

— **Fiabilité électronique :**
  — Amélioration de la gestion des câbles et sécurisation des connecteurs (soudure ou fixations mécaniques) pour prévenir les déconnexions dues aux vibrations.
  — Équilibrer davantage le couple des moteurs ou utiliser des moteurs plus homogènes.

3 Partie informatique

Cette seconde partie décrit la démarche logicielle adoptée dans le cadre du cycle en V : analyse du problème, conception des modules, choix des algorithmes, puis implémentation et tests. Deux programmes communicants sont réalisés : un planificateur qui lit le fichier texte de description de la ville et produit une suite d'ordres, un pilote qui lit ces ordres sur son entrée standard et contrôle le robot en temps réel tout en affichant les informations sur l'écran LCD et en émettant un signal sonore sur les pastilles.

3.1 Analyse

3.1.1 Analyse Descendante

L'architecture du projet a été conçue selon une approche descendante (Top-Down) pour isoler les responsabilités et faciliter les tests. Le système est décomposé en niveaux hiérarchiques, comme illustré par le diagramme suivant qui reprend l'ensemble des fonctions implémentées.

Le diagramme d'analyse descendante pour la partie planification en annexe.

Les niveaux sont :

— **Niveau orchestration (Niveau 1)** : Programme principal et gestion du cycle de vie.

— **Niveau métier (Niveau 2)** : Algorithmes de chargement (Parsing) et de planification (TSP).

— **Niveau noyau (Niveau 3)** : Algorithme de recherche de chemin A*.

— **Niveau données (Niveau 4)** : Opérations élémentaires sur les Types Abstraits de Données (TAD).

3.2 Conception Préliminaire

Cette phase définit les TAD utilisés pour modéliser le problème, en correspondance directe avec l'implémentation C.

3.2.1 TAD Case

Correspondance code C : `unsigned int` (identifiant de noeud).

Paramètres

— `id : Naturel`

Utilise

— Entier

Opérations

— `creerCase : Entier → Case`

— `estEgal : Case × Case → Booléen`

— `valeur : Case → Entier`

Axiomes
- `valeur(creerCase(i)) = i`
- `estEgal(c1, c2) = (valeur(c1) == valeur(c2))`

3.2.2 TAD Direction

Correspondance code C : `enum Direction { Nord=0, Est=1, Sud=2, Ouest=3 }`.

Paramètres
- `dir : Entier` (0..3)

Utilise
- Entier, Booléen

Opérations
- `Nord, Est, Sud, Ouest : → Direction`
- `tournerDroite : Direction → Direction`
- `tournerGauche : Direction → Direction`
- `opposee : Direction → Direction`
- `directionVersEntier : Direction → Entier`
- `entierVersDirection : Entier ⇸ Direction`

Axiomes
- `tournerDroite(Nord) = Est`, `tournerDroite(Est) = Sud`, `tournerDroite(Sud) = Ouest`, `tournerDroite(Ouest) = Nord`
- `tournerGauche(d) = opposee(tournerDroite(d))`
- `opposee(Nord) = Sud`

3.2.3 TAD Labyrinthe

Correspondance code C : Structure `CityConfig` contenant un `G_Graphe`.

Paramètres
- `largeur, hauteur : Entier`
- `graphe : Graphe`
- `checkpoints : Liste<Case>`
- `depart : Case`, `directionDepart : Direction`

Utilise
- Entier, Case, Direction, Graphe, Liste

Opérations
- `creerLabyrinthe : Entier × Entier × Graphe × ... → Labyrinthe`
- `obtenirLargeur, obtenirHauteur : Labyrinthe → Entier`
- `estConnecte : Labyrinthe × Case × Case → Booléen`
- `caseVersCoord : Labyrinthe × Case → (x:Entier, y:Entier)`
- `coordVersCase : Labyrinthe × Entier × Entier ⇸ Case`

Axiomes
- `estConnecte(l, c1, c2) = G_arcPresent(l.graphe, c1, c2)`
- `caseVersCoord : ID = y * largeur + x + 1`

3.2.4 TAD Instruction (Action)

Correspondance code C : `enum ActionType { AV=0, TD=1, TG=2 }`.

Opérations
- `AV, TD, TG : → ActionType`

3.2.5 TAD ListeInstructions (Chemin)

Correspondance code C : Tableau de `Step` ou Liste Chaînée.

Utilise
- Instruction, Case, Entier

Opérations
- `creerListeVide : → ListeInstructions`
- `ajouterEnFin : ListeInstructions × Instruction × Case → ListeInstructions`
- `concatener : ListeInstructions × ListeInstructions → ListeInstructions`
- `longueur : ListeInstructions → Entier`

Axiomes
- `longueur(concatener(l1, l2)) = longueur(l1) + longueur(l2)`

3.2.6 Partie Robot

Pour le robot, nous n'avons pas de TAD. Cependant, nous avons réalisé une analyse descendante des fonctions et procédures requises pour la gestion du robot. Nous allons seulement aller en détail sur la gestion des instructions car c'est la partie logique du robot. Le schéma se trouver en annexe de ce rapport. L'ensemble des pseudocodes des fonctions est dans `programmes/utils/pseudocodes_repartition.md` sur notre dépôt GitLab. Nous présenterons tout de même le programme principal, la gestion des intersections pour la partie robot et le suivi de ligne.

3.3 Conception détaillée et algorithmes

Cette section détaille les algorithmes majeurs (Intelligence et Logique du robot).

3.3.1 Algorithme A* (Espace d'Etats)

Optimisation clé est la recherche dans l'espace $(Noeud, Direction)$ pour inclure le coût des rotations.

```
Structure State
    node: Entier
    dir: Direction
FinStructure
```

```
Fonction astar_search(graphe, start_node, start_dir, target_node) : Liste<Action>
    OpenSet <- FilePriorite()
    ClosedSet <- Tableau[NOEUDS][4] initialise a INFINI

    start_state <- State(start_node, start_dir)
    OpenSet.ajouter(NodeData(start_state, 0, heuristique(start_node, target_node)))
    ClosedSet[start_node][start_dir] <- 0

    Tant que non OpenSet.estVide() faire
        current <- OpenSet.extraireMin()
        // Si on atteint le noeud cible (peu importe la direction finale)
        Si current.state.node == target_node alors
            Retourner reconstruire_chemin(current)
        FinSi

        // 1. Action AVANCER
        voisin <- get_voisin_devant(graphe, current.state.node, current.state.dir)
        Si voisin existe alors
            new_g <- current.g_score + COST_MOVE
            Si new_g < ClosedSet[voisin][current.state.dir] alors
                ClosedSet[voisin][current.state.dir] <- new_g
                next_state <- State(voisin, current.state.dir)
                OpenSet.ajouter(NodeData(next_state, new_g, new_g + h(voisin), current.state, AV))
            FinSi
        FinSi

        // 2. Action TOURNER DROITE
        dir_droite <- (current.state.dir + 1) mod 4
        new_g_rot <- current.g_score + COST_TURN
        Si new_g_rot < ClosedSet[current.state.node][dir_droite] alors
             ClosedSet[current.state.node][dir_droite] <- new_g_rot
             next_state <- State(current.state.node, dir_droite)
             OpenSet.ajouter(NodeData(next_state, new_g_rot,new_g_rot+h(current.state.node),current.state,TD))
        FinSi

        // 3. Action TOURNER GAUCHE
        dir_gauche <- (current.state.dir + 3) mod 4
        new_g_gauche <- current.g_score + COST_TURN
        Si new_g_gauche < ClosedSet[current.state.node][dir_gauche] alors
            ClosedSet[current.state.node][dir_gauche] <- new_g_gauche
            next_state <- State(current.state.node, dir_gauche)
            OpenSet.ajouter(NodeData(
                next_state,
                new_g_gauche,
                new_g_gauche+h(current.state.node),
                current.state,TG)
            )
        FinSi
    FinTantQue
    Retourner ECHEC
Fin
```

3.3.2 Voyageur de Commerce (TSP) Hybride

On utilise une stratégie hybride pour visiter les checkpoints de manière optimale.

```
Fonction solve_tsp(graphe, start_node, start_dir, checkpoints) : Liste<Action>
    Si longueur(checkpoints) <= 10 alors
        // Approche Exhaustive
        best_cost <- INFINI
        best_path <- VIDE
        Pour chaque permutation P de checkpoints faire
            (cost, path) <- evaluer_sequence(start_node, start_dir, P)
            Si cost < best_cost alors
                best_cost <- cost
                best_path <- path
            FinSi
        FinPour
        Retourner best_path
    Sinon
        // Approche Gloutonne
```

```
        curr_node <- start_node
        curr_dir <- start_dir
        path_total <- Vide
        Tant que checkpoints non visites faire
            best_next_cp <- trouver_plus_proche_astar(curr_node, curr_dir, checkpoints)
            segment <- astar_search(graphe, curr_node, curr_dir, best_next_cp)
            path_total += segment
            (curr_node, curr_dir) <- fin(segment)
            Retirer best_next_cp de checkpoints
        FinTantQue
        // Retour au depart
        path_total += astar_search(graphe, curr_node, curr_dir, start_node)
        Retourner path_total
    FinSi
Fin
```

3.3.3 Parsing du Labyrinthe

Chargement de la configuration depuis un fichier texte.

```
Fonction load_city(filename) : CityConfig
    config.graphe <- NouveauGraphe()

    Lire N (taille)
    Lire StartNode, StartDir

    Tant que non EOF faire
        Si ligne est "U-V" alors
            AjouterArete(config.graphe, U, V)
        Sinon Si ligne est "N" alors
            AjouterCheckpoint(config.checkpoints, N)
        FinSi
    FinTantQue

    Retourner config
Fin
```

3.4 Programme principal du robot

```
fonction principale main() : entier
    debut
        debut_temps <- temps_actuel_millis()

        setupPin()
        lcd_display_init()

        nb_instr <- 0

        creer_thread(thread_RGB, rgb_color_thread_loop, NULL)

        instructions_robot <- charger_instructions_dynamique(stdin, nb_instr)
        ecrire("J'ai recu " + nb_instr + " instructions.")

        pour i de 0 a nb_instr-1 faire
            ecrire("Instruction " + i + " : " + instructions_robot[i])

            si instructions_robot[i] = "AV" alors
                instr <- 0
                afficher_LCD(1,"Avancer")
            sinon si instructions_robot[i] = "TG" alors
                instr <- 1
                afficher_LCD(1,"Gauche")
            sinon si instructions_robot[i] = "TD" alors
                instr <- 2
                afficher_LCD(1,"Droite")
            sinon
                ecrire("Instruction inconnue: " + instructions_robot[i])
                stop()
```

```
                continuer
            fin si

            gestionIntersection(instr)
        fin pour

        // Liberation memoire
        pour i de 0 a nb_instr-1 faire
            liberer(instructions_robot[i])
        fin pour
        liberer(instructions_robot)

        afficher_LCD(0,"Circuit")

        fin_temps <- temps_actuel_millis()
        duree <- (fin_temps - debut_temps)/1000.0  // conversion en secondes
        afficher_LCD(1, convertir_en_chaine(duree) + " s")

        ecrire("Duree totale : " + convertir_en_chaine(duree) + " secondes")

        attendre_thread(thread_RGB)
        stop()

        retourner 0
    fin
```

3.5 Programme de gestion des intersections

```
procedure gestionIntersection(instr : entier)
    debut
        selon instr faire
            cas 0 : gestionSuiviLigne(DELAIS_INTERSECTION)
            cas 1 : gaucheIntersection(DELAIS_INTERSECTION)
            cas 2 : droiteIntersection(DELAIS_INTERSECTION)
            autre : ecrire("Instruction invalide")
        fin selon
    fin
```

3.6 Programme de suivi de ligne

```
procedure gestionSuiviLigne(delais : entier)
    debut
        avance()
        tant que NON surIntersection() faire
            si lecturePin(CAPTEUR_LIGNE_GAUCHE) ET NON lecturePin(CAPTEUR_LIGNE_MILIEU) alors
                gauche()
            sinon si lecturePin(CAPTEUR_LIGNE_DROIT) ET NON lecturePin(CAPTEUR_LIGNE_MILIEU) alors
                droite()
            sinon
                avance()
            fin si
            attendre(delais)
        fin tant que
    fin
```

3.7 Implémentation et tests

L'implémentation respecte la structure standard C.

3.7.1 Tests unitaires

Une suite de tests unitaires a été développée à l'aide du framework **CUnit**. Ces tests sont situés dans le répertoire `programmes/src/algo` et se basent sur les axiomes définis

dans les spécifications des TAD.

Les tests couvrent les modules suivants :

— **TAD Labyrinthe** (`test_labyrinthe.c`) : Vérifie le chargement correct des dimensions, la position initiale et la connectivité du graphe (fonction `estConnecte`). Il valide également les axiomes de conversion de coordonnées (`caseVersCoord`).

— **TAD Instruction** (`test_instructions.c`) : Valide la structure des données `Step` et les énumérations `ActionType` et `Direction`.

— **Graphe** (`test_graphe.c`) : Teste les opérations de base du graphe générique : création, ajout et suppression de sommets et d'arcs, ainsi que la récupération des sommets adjacents.

— **Algorithme A*** (`test_astar.c`) : Valide la recherche de chemin sur des graphes simples (grilles) et vérifie le comportement de l'algorithme dans des cas limites (chemin inexistant).

3.7.2 Exemple de validation

Pour valider le fonctionnement de l'algorithme de planification sur un cas concret, nous présentons ici un exemple d'exécution complet. Cette validation permet de confirmer que le chemin généré respecte bien les contraintes du labyrinthe (murs, sens de rotation) et visite tous les checkpoints demandés.

Espace de test (fichier d'entrée) Le fichier descriptif ci-dessous définit une grille de 5x5 avec des murs et 3 checkpoints à visiter (13, 22, 20).

```
5
6S
1-2
1-6
2-3
2-7
3-4
4-5
5-10
6-11
7-8
8-13
9-14
10-15
11-16
11-12
12-17
12-13
14-19
15-20
16-21
17-22
18-19
18-23
19-20
20-25
21-22
22-23
23-24
13
22
20
.
```

Résultat (instructions générées) Le planificateur produit la séquence d'instructions suivante, qui mène le robot successivement aux points 13, 22 et 20 avant de revenir au départ.

```
AV
TG
AV
AV
AV
TD
AV
TG
AV
TD
AV
TD
AV
AV
AV
TD
AV
AV
.
```

Visualisation La figure ci-dessous illustre graphiquement le chemin calculé par l'algorithme au sein du labyrinthe, confirmant le passage par les checkpoints.

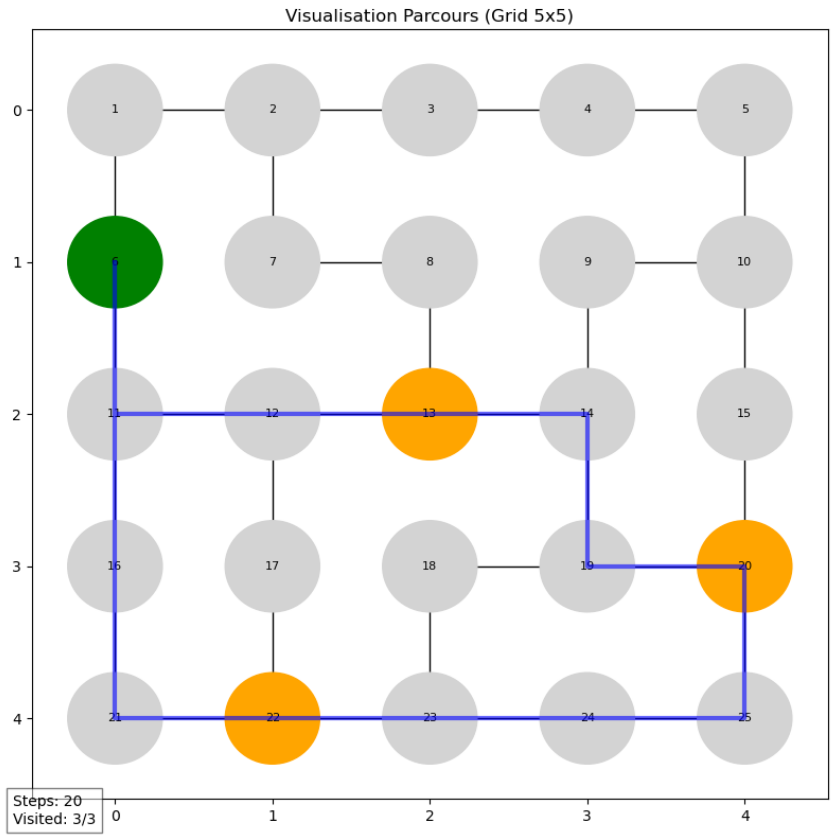


FIGURE 3 – Visualisation du chemin calculé pour l'exemple de validation

4 Conclusion et matrice de contribution

Ce projet intégratif a constitué un véritable défi, notamment en raison du temps imparti. Nous ne sommes pas les seuls à avoir prolongé le travail en dehors des séances afin d'accomplir les attendus du projet. Cependant, ce fut une expérience enrichissante pour l'ensemble des membres du groupe, tant au niveau du travail d'équipe que pour la mise en pratique des différentes notions abordées au cours de ce semestre.

D'un point de vue organisationnel, l'usage d'outils de gestion de projet plus détaillés, comme les diagrammes de Gantt ou des tableaux Kanban, aurait permis d'assurer un meilleur suivi de l'avancement du projet et d'anticiper les points à approfondir.

Finalement, le passage de notre robot le jour de notre robot. Celui-ci n'a pas pu correctement effectuer les actions du planificateur, en raison d'un problème lié à la détection des intersections, bogue que l'on associe aux capteurs. Comme présenté précédemment, la solution envisagée pour ce bogue est un protocole de calibration avant le parcours du robot. Par ailleurs, un problème de planification a été observé durant le passage. Les solutions envisagées pour y remédier incluent une augmentation du nombre de tests unitaires et avec assez de temps, une preuve de l'algorithme, par exemple via la programmation logique.

4.1 Résumé des contributions

Pour lire la matrice des contributions, on considère :
— O : Oui
— N : Non

Tâche	Fakhoury	Grondin	Kamara	Saad Djaballah	Savary
Analyse					
Electronique	N	N	N	O	N
Informatique					
Logique Robot	N	O	N	N	O
Composants	O	N	N	N	O
Planificateur	O	N	O	N	N
Tests/Intégration					
Electronique	N	N	N	O	N
Informatique	O	O	O	N	O
Documentation					
Electronique	N	N	N	O	N
Informatique	O	O	O	N	O
Rédaction Rapport					
Electronique	N	N	N	O	O
Informatique					
Logique Robot	N	O	N	N	N
Planificateur	N	N	O	N	N
Forme & relecture	O	O	N	N	N

Table 2 – Matrice de contribution des membres du projet.

5 Annexe du rapport

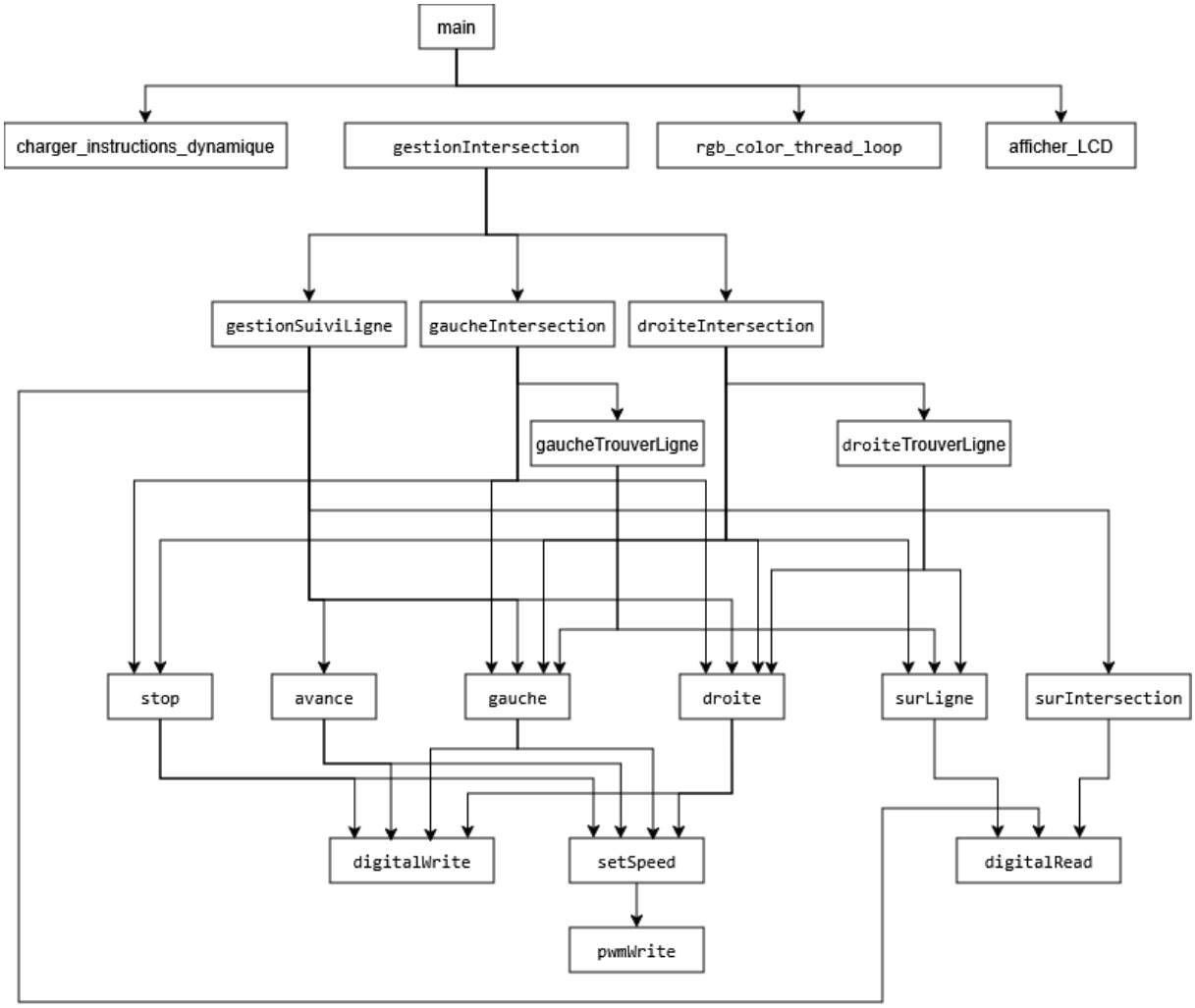


FIGURE 4 – Diagramme d'analyse descendante pour les programmes du robot

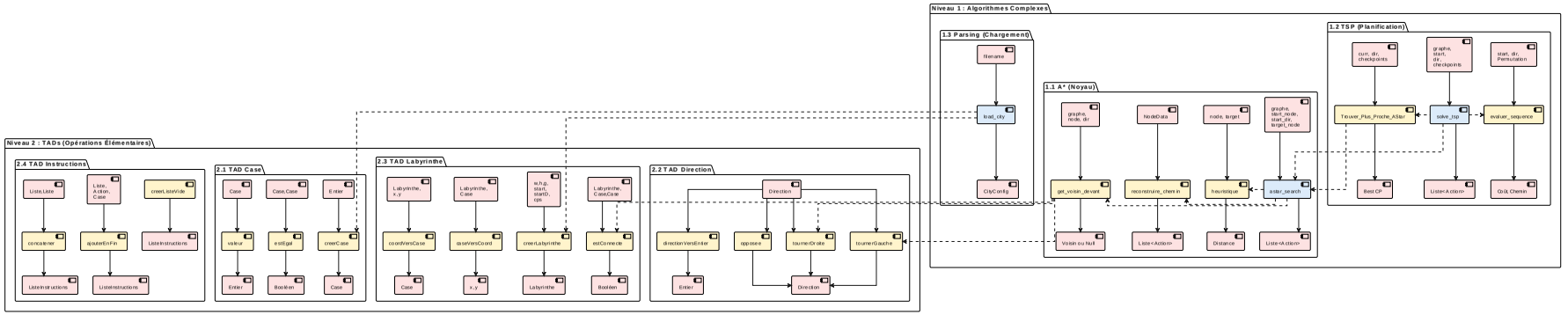


FIGURE 5 – Diagramme d'analyse descendante pour la planification